

Part 1: Origin, Velocity & Slope

1. "Walk me through your background and how you got into the AI space."

- **The Intent:** Assess technical direction, velocity, and whether you are a passive student or an active builder.
- **The Answer:** "I'm a graduate student in Artificial Intelligence at Northeastern, but my focus has always been on moving models out of research sandboxes and into production cloud containers. To give myself a dedicated infrastructure environment to deploy research-grade machine learning models to the cloud, I independently launched a project architecture named Arxelos. Through building custom Vision-Language-Action pipelines and medical-grade data retrieval systems, I realized that the real bottleneck in AI isn't model weights—it's building type-safe, resilient infrastructure around probabilistic models. That's what brought me squarely into the applied AI engineering space."

2. "Why Aegis? Of all the AI startups out there, why are you interested in healthcare Revenue Cycle Management (RCM)?"

- **The Intent:** Verify mission alignment and ensure you understand the complex, painful economic problem they are solving.
- **The Answer:** "I'm drawn to high-stakes, asymmetric spaces where software quality directly impacts human lives. In standard B2B SaaS, an engineering bug means a dropped dashboard metric. In healthcare RCM, an unhandled error or a hallucinated denial appeal means a hospital doesn't get paid and a real patient faces severe financial toxicity. I've seen how this administrative friction forces people to ration critical blood pressure and diabetes medications because unexpected medical billing loops skyrocket out of network. I want to work at Aegis because building accurate, hyper-resilient agent automation to beat insurance company denial algorithms is an engineering challenge with a direct, ethical mission."

3. "Our stack is heavily built on TypeScript and Node.js, but your background shows a lot of Python. How do you feel about transitioning to our stack?"

- **The Intent:** Test adaptability and whether you understand how language runtimes affect application performance.
- **The Answer:** "I see Python and TypeScript as tools for fundamentally different jobs. Python is unmatched for mathematical matrix manipulation, model training, and data science. But for building **Agent Runtimes**, TypeScript is structurally superior. AI agents are heavily I/O-bound—they spend 90% of their execution cycles waiting on asynchronous network requests from LLM endpoints, EHR databases, and payer portal scrapers. Node.js's non-blocking, single-threaded event loop handles thousands of concurrent I/O connections flawlessly without heavy context-switching overhead. Layering TypeScript over that runtime gives us compile-time data contracts that eliminate null-pointer crashes before shipping code."

4. "What does 'High Agency' mean to you, and how have you demonstrated it recently?"

- **The Intent:** Evaluate cultural fit for a lean, fast-moving YC startup where developers must operate independently.
- **The Answer:** "To me, High Agency means that when you hit a wall, you don't file a report and wait for direction—you find a way over, under, or through it independently. Recently, while deploying a custom deep learning pipeline, I hit an undocumented mismatch between my local environment's tensor configurations and the remote cloud container runtime. Instead of stepping back or waiting for a tutorial update, I dissected the underlying deployment configurations, isolated the package dependency version delta, rebuilt the base Docker configurations locally, and forced compilation stability myself. High Agency is the refusal to let operational friction slow down your execution momentum."

Part 2: Technical Moats & System Architecture

5. "Tell me about the most complex AI pipeline you've built independently. Where did it break, and how did you fix it?"

- **The Intent:** Uncover real debugging scars, system design instincts, and engineering depth.
- **The Answer:** "I built an advanced Medical RAG pipeline to spirit insights from unstructured clinician records. The initial failure state I ran into was **semantic text dilution**. When embedding large blocks of text, tiny but critical clinical facts—like a specific notation that a patient completed 6 weeks of prerequisite conservative physical therapy—got mathematically drowned out in the vector space, leading to inaccurate generation context. To fix it, I refactored the backend into a **Hierarchical Parent-Child RAG architecture**. I split the charts into highly granular, 3-sentence child chunks for precise vector alignment, but configured the database engine to return the macro parent document section to the LLM context layer. This maintained localized data accuracy while preserving overall structural clinical context."

6. "In our domain, a hallucinated medical code can result in a claim being permanently rejected. How do you mathematically guarantee an LLM agent doesn't invent or hallucinate a medical code?"

- **The Intent:** Test practical AI safety boundaries and see if you rely on loose prompts or hard code boundaries.
- **The Answer:** "We can't treat LLMs as database lookups. To achieve a 0% code hallucination rate, we must completely separate code selection from text generation. My approach is to query the hospital's EHR or the primary 837 claim file deterministically using standard database APIs first. We extract the legally valid CPT or ICD-10 codes and lock them into the prompt environment as immutable 'Anchor Variables.' We instruct the model that it is structurally forbidden from referencing any token outside that explicit

array. Finally, we intercept the string output using a Zod runtime validation shield and regex filters to confirm code integrity before the appeal data ever hits a human dashboard."

7. "If a hospital client drops 10,000 multi-step denial reviews into our platform simultaneously, how do you prevent our background agents from crashing our core databases or getting throttled by external insurance endpoints?"

- **The Intent:** Assess cloud architecture knowledge, handling of backpressure, and third-party API limits.
- **The Answer:** "You cannot process high-volume, long-running agent tasks on a linear first-in, first-out sequence. I would decouple ingestion from execution. Incoming denial events should hit an asynchronous **AWS SQS FIFO queue** to serve as a traffic buffer. We then deploy a worker pool hosted on **AWS ECS Fargate containers**, allowing the agents to pull tasks at a controlled, scalable pace. If an external insurance portal returns a **429 Too Many Requests** rate limit midway through a 10-step agent loop, the agent shouldn't crash or restart from scratch. It saves its state to a persistent **Redis Checkpointer** and fires an **exponential backoff with random jitter** to clear the rate limit smoothly without flooding the event loop."

8. "Hospitals drop thousands of denied claims into our dashboard at once. How would you architect an intelligent denial prioritization queue in Node.js?"

- **The Intent:** Evaluate resource management and your ability to map business rules directly into database and queue logic.
- **The Answer:** "A first-in, first-out queue fails business needs because a \$50,000 surgical denial with a 48-hour filing deadline gets stuck behind a \$150 routine test. I would write a worker process that intercepts the SQS queue and assigns a dynamic priority score using a composite formula:
$$\text{Priority Score} = (\text{Financial Deficit}) \times (\text{Overturn Probability}) \cdot e^{-\lambda t}$$

Where t represents the remaining time until the insurance timely-filing deadline, and λ is a decay constant scaling urgency. The worker continuously updates this score in a Redis sorted set, ensuring our ECS Fargate agents always pull the highest-value, highest-urgency tasks first."

9. "How do you test your agent architectures? What does your evaluation pipeline look like before a prompt or agent change goes live?"

- **The Intent:** Test LLMOps maturity and understand how you evaluate non-deterministic AI outputs.
- **The Answer:** "We cannot guess-and-check prompts in production. I build automated evaluation loops that run against a **Golden Dataset** of historically verified denials and

successful human appeals. Every time an agent configuration or prompt changes, the CI/CD pipeline triggers an automated testing run. We use a secondary LLM-as-a-judge configuration to grade the output text across three strict, mathematical metrics:

Faithfulness (ensuring no facts outside the retrieved EHR are invented), **Answer Relevance** (ensuring it directly refutes the CARC code), and **Schema Compliance** via Zod. If any metric drops below a 95% threshold, the deployment circuit trips and blocks the merge."

10. "What happens if our agent submits an appeal, and it gets denied again? How do you architect a multi-turn appeal tracking system?"

- **The Intent:** Evaluate state-machine design skills and your ability to manage complex, long-running asynchronous lifecycles.
- **The Answer:** "We must treat the appeal process as a persistent state machine rather than a single event transaction. I model this using a relational database schema tracked by a strict `WorkflowStatus` Enum: `INGESTED` $\rightarrow$ `EHR_PROCESSING` $\rightarrow$ `APPEAL_GENERATED` $\rightarrow$ `SUBMITTED` $\rightarrow$ `RE_DENIED` $\rightarrow$ `ESCALATED`. If a secondary 835 file marks a previously appealed claim as denied again, it transitions the state to `RE_DENIED`. This state change triggers a specialized agent workflow that extracts the secondary denial reason, links it to the historical trace of the first appeal, and automatically routes the file to a senior human reviewer or a higher-tier legal appeal layout."

Part 3: Data Integrity & Production AI

11. "A massive volume of insurance denials arrive as messy, multi-page PDFs, scans, or physical faxes. How would you approach building an ingestion pipeline that reliably extracts tabular data without losing cross-page context?"

- **The Intent:** See how you handle the real-world, unglamorous data fragmentation that defines healthcare software.
- **The Answer:** "Parsing multi-page medical financial documents is an extraction and alignment problem. If you simply throw raw OCR text chunks at an LLM, table layouts break across page cuts, disconnecting line items from their headers. My approach is a multi-modal split pipeline. First, we use a document layout parser to preserve visual coordinates and convert pages into a structured Markdown array rather than raw text blocks. Second, we apply a sliding context window during chunking where every single snippet retains an immutable metadata header detailing the unique Document ID, Patient Token, and overall claim aggregates. This ensures that when our agent parses an adjustment code on page 5, the context natively binds it back to the entry charges on page 1."

12. "Running complex, multi-turn agent loops over frontier models gets incredibly expensive and high-latency very quickly. How do you optimize to balance high clinical accuracy with low token consumption?"

- **The Intent:** Check if you treat API budgets responsibly and understand cost/performance optimization.
- **The Answer:** "We can't solve latency and cost by simply down-grading models globally, because clinical accuracy degrades. Instead, I deploy a **Router-Worker pattern**. For deterministic tasks—like extracting static insurance IDs, parsing dates, or stripping boilerplate text from an EOB—we route the text to lightweight models or clean TypeScript regex routines, which resolve in milliseconds for a fraction of a cent. We save our expensive frontier model tokens exclusively for the final orchestration node: synthesizing the extracted evidence into a persuasive, policy-compliant legal appeal letter. By minimizing the context prefix through aggressive RAG filtering beforehand, we maximize accuracy while protecting our margins."

13. "We have a lot of unstructured text in EHRs. How do you approach cleaning or filtering out noise before passing it to an embedding model?"

- **The Intent:** Evaluate data preprocessing discipline. Garbage in, garbage out in RAG systems.
- **The Answer:** "Raw EHR data is full of noise—system boilerplate, redundant headers, signatures, and empty diagnostic templates that warp vector embeddings. Before chunking, the text passes through a deterministic text cleaning pipeline. We use regex and token filters to strip out repetitive UI layout text, boilerplate legal disclaimers, and redundant system timestamps. Furthermore, we leverage semantic filtering: if an entire section of a note is completely blank or contains generic template text with no patient-specific variables filled in, we discard it entirely. This keeps our vector database pure, improves embedding alignment, and saves downstream token costs."

14. "Our platform is built to be used by hospital billing specialists, not software engineers. How do you approach building backend systems so that a non-technical user can easily trust and interact with the AI?"

- **The Intent:** Assess product empathy and your ability to make complex AI systems transparent and controllable.
- **The Answer:** "An AI agent shouldn't operate as an opaque black box; it needs to function as a collaborative partner. When our backend generates an appeal draft, the JSON response shouldn't just contain raw text—it must include explicit **Citations and Confidence Scores**. When a billing specialist opens the document in our UI, they shouldn't have to guess why a paragraph was written. The system visually highlights the exact sentence in the hospital's clinical notes that justified the argument. If the agent outputs a low confidence index due to an ambiguous payer rule, our API surfaces that specific clause as an editable field. This turns the user into a high-level editor, building deep structural trust in our automation."

Part 4: Startup Economics, Strategy & Team Fit

15. "At an early-stage startup, we are constantly balancing shipping a feature in two hours to unblock a client versus spending two days building a pristine architecture. How do you navigate the tension between 'scrappy hacking' and 'long-term engineering'?"

- **The Intent:** Ensure you aren't over-engineering or letting academic perfection block product shipment speed.
- **The Answer:** "I look at code implementation as a temporary experiment, but data models and interfaces as permanent commitments. My framework is to be completely scrappy inside the function logic, but strict about the boundary contracts. If we need to quickly integrate a new niche payer portal, I will write a straightforward, pragmatic script to pull the data and unblock the user immediately. However, I will ensure that the script outputs data that conforms exactly to our centralized TypeScript interfaces and passes through a Zod schema. By keeping the input/output boundaries rigid, the internal code remains modular. When the feature proves its value, we can refactor the backend mechanics completely without causing regression ripples across the rest of our pipelines."

16. "We move incredibly fast, and things change daily. How do you deal with ambiguity or shifting priorities?"

- **The Intent:** Assess resilience under the standard chaos of a pre-series A technology startup.
- **The Answer:** "I look at code as a depreciating asset and architectural logic as the permanent foundation. When priorities pivot, I don't get defensive about discarded code. I instantly strip the system down to its data models and contracts. If we need to scrap a portal scraping approach tomorrow because an insurance carrier updated their security wall, my immediate move isn't to panic—it's to check our state checkpoints, ensure our data models are decoupled from the delivery layer, and look for an alternative endpoint standard like a FHIR API. I thrive when the target moves, as long as we maintain strict boundaries around our types, data integrity, and safety."

17. "How do you deal with rate limits or structural dropouts when building scrapers for payer portals that actively try to block automated systems?"

- **The Intent:** Evaluate practical problem-solving skills when dealing with hostile, uncooperative external platforms.
- **The Answer:** "Payer portals are highly volatile and actively deploy anti-bot systems. To build resilient scraping layers inside our ECS Fargate containers, we cannot rely on fragile browser scripts. First, we wrap all portal browser actions (using frameworks like Playwright) in a strict **Circuit Breaker pattern**. If a portal blocks our connection 5 times sequentially, the circuit trips *Open*, stops wasting resources, and alerts our system to fail

fast. Second, we decouple the browser session: we route traffic through rotating residential proxies and inject random human-like mouse paths and variable delays. Finally, if a portal updates its UI layout and breaks our scraper, our Zod validation shield catches the parsing failure instantly, alerts the team via an automated telemetry trace, and cleanly routes that single claim to a manual queue while the rest of the engine keeps running."

18. "Aegis was founded by close friends who trust each other implicitly and move at extreme speeds. In a lean startup environment under intense pressure, how do you handle internal technical disagreements or personal friction?"

- **The Intent:** Evaluate ego, emotional maturity, and communication style within a small, tight-knit room.
- **The Answer:** "In a fast-moving room, friction is healthy as long as it's driven by product obsession rather than personal ego. My framework for resolving technical disagreements is to completely remove opinions and default to quick, measurable validation experiments. If we disagree on whether a specific RAG chunking strategy or database index will perform better under load, we don't spend hours debating it in a meeting. We jump into our sandbox container setup, write two quick test routines, run them against our evaluation dataset, and let the latency metrics and accuracy logs make the decision for us. Once the data gives us an answer, we commit to the winning path 100% and keep moving forward. The target is winning the market, not winning the argument."

19. "As an AI startup handling real hospital accounts, HIPAA compliance is non-negotiable. How do you ensure patient data privacy when using third-party LLM APIs?"

- **The Intent:** Confirm you understand the legal and technical architectures required to protect Protected Health Information (PHI).
- **The Answer:** "Data security has to be handled locally before any network packet leaves our cluster. I look at this as a two-stage **Anonymization and Re-identification Proxy pipeline**. Before any clinical note text is sent to an external LLM API, the payload passes through a local Named Entity Recognition (NER) script to scrub PHI—like names, explicit geographic IDs, and exact dates. We swap these sensitive records out for unique cryptographic hash placeholders, such as `[PATIENT_TOKEN_A1]`. The external model processes the clean, anonymized clinical argument and returns the appeal text. Our Node.js backend then performs a deterministic, localized re-injection of the original data fields right before writing it to our secure PostgreSQL instance, guaranteeing outside APIs never process exposed identities."

20. "What is your current timeline, and what are your expectations for joining our team?"

- **The Intent:** Lock down logistics, commitment boundaries, and structural availability.
- **The Answer:** "I want to be completely transparent and proactive about my timeline. I am currently finishing an intense academic block focusing on advanced computing foundations. Because I believe in giving 100% of my bandwidth to a single arena, I am executing this preparation block now so I can relocate to San Francisco the first week of August. While typical summer interns walk out the door in mid-August right when they become operationally valuable, I am explicitly positioning myself for a **Fall engagement**. This gives Aegis deep operational continuity. I will land in the Bay Area completely un-distracted by classes, fully native to your TypeScript and AWS stack, and ready to provide uninterrupted engineering leverage through your entire autumn product scaling cycle."